

Laboratorio: Construcción paso a paso de un sistema RAG en Python para conversar con documentos, incorporar voz y explorar grafos semánticos

En este laboratorio los estudiantes por grupos de trabajo desarrollarán, de manera secuencial y guiada, una solución básica de inteligencia artificial documental basada en arquitectura RAG (Retrieval-Augmented Generation). A lo largo de la práctica comprenderá cómo cargar documentos, fragmentarlos, convertirlos en embeddings, almacenarlos en una base vectorial y recuperar contexto relevante para responder preguntas fundamentadas en el contenido. Como componente complementario, explorará la incorporación de voz y la construcción inicial de grafos semánticos a partir de entidades y relaciones presentes en los documentos.

Para apoyar el proceso, el laboratorio incorpora un prompt guía explícito que servirá como tutor técnico durante cada fase del desarrollo. El propósito no es solo ejecutar código, sino comprender con rigor la arquitectura, justificar las decisiones de diseño y analizar críticamente el comportamiento del sistema construido.

1. Datos generales del laboratorio

Nombre del laboratorio:

Construcción de un sistema RAG en Python para consulta inteligente de documentos por chat y voz, con exploración de grafos semánticos

Área:

Inteligencia Artificial Aplicada / Recuperación Aumentada por Generación (RAG)

Modalidad:

Práctico guiado

Nivel:

Intermedio

Duración estimada:

6 a 8 horas, distribuidas por fases

Herramientas base:

Python, Jupyter o Google Colab, PyMuPDF, sentence-transformers, FAISS o Chroma, NetworkX o PyVis, motor LLM, librerías de voz

2. Propósito del laboratorio

Este laboratorio tiene como propósito guiar al estudiante en la construcción progresiva de un sistema basado en la arquitectura **RAG (Retrieval-Augmented Generation)**, capaz de leer documentos, fragmentarlos, representarlos mediante embeddings, almacenarlos en una base vectorial y responder preguntas fundamentadas en su contenido. Como ampliación, el estudiante incorporará una capa de interacción por voz y una aproximación inicial a la representación del conocimiento mediante grafos semánticos.

El laboratorio no se limita a ejecutar código: busca que el estudiante comprenda la lógica arquitectónica del sistema, identifique la función de cada componente y desarrolle criterio técnico para evaluar las decisiones de diseño adoptadas en la solución.

3. Resultado de aprendizaje

Al finalizar este laboratorio, el estudiante estará en capacidad de:

- comprender la arquitectura general de un sistema RAG;

- construir paso a paso un flujo de ingesta, fragmentación, vectorización y recuperación documental;
- implementar una consulta básica tipo chat sobre documentos;
- reconocer cómo integrar entrada o salida por voz en un sistema documental conversacional;
- identificar entidades y relaciones para construir una base inicial de grafo semántico;
- documentar el proceso técnico realizado y justificar las decisiones adoptadas.

4. Competencias que fortalece

- Pensamiento computacional aplicado a IA.
- Diseño de arquitecturas de recuperación semántica.
- Implementación de flujos NLP en Python.
- Capacidad de análisis crítico sobre recuperación, contexto y generación.
- Comprensión de la trazabilidad documental en sistemas basados en LLM.

5. Metodología de trabajo

El laboratorio se desarrollará de forma **secuencial y guiada**. Cada estudiante deberá construir el sistema por etapas, verificando el funcionamiento de cada componente antes de pasar al siguiente. No se debe avanzar al siguiente paso si el anterior no ha sido comprendido y validado.

La lógica del trabajo será:

1. comprender la arquitectura;
2. preparar el entorno;
3. cargar documentos;
4. fragmentar el contenido;
5. generar embeddings;
6. almacenar en base vectorial;
7. recuperar información relevante;
8. construir respuestas apoyadas en contexto;
9. ampliar con voz;
10. explorar grafos semánticos.

6. Prompt guía explícito para apoyar al estudiante

Este prompt debe entregarse al estudiante de forma visible dentro del laboratorio. La idea es que pueda usarlo como apoyo conceptual y técnico mientras desarrolla cada fase.

Prompt guía del laboratorio

Actúa como tutor experto en arquitectura RAG, Python y sistemas de recuperación documental con LLM.

Estoy desarrollando un laboratorio paso a paso para construir un sistema RAG que permita conversar con documentos y, posteriormente, integrar voz y grafos semánticos.

Necesito que me ayudes bajo estas reglas:

1. Explícame cada paso de manera clara, técnica y pedagógica.
2. Antes de dar código, indícame cuál es el propósito del paso.
3. Si me das código, debe estar comentado y ser funcional en Python.

4. Debes decirme qué entrada recibe el componente, qué proceso realiza y qué salida produce.
5. Debes advertirme errores comunes y cómo verificarlos.
6. Debes mantener coherencia con una arquitectura RAG compuesta por:
 - carga de documentos,
 - limpieza y fragmentación,
 - embeddings,
 - base vectorial,
 - retrieval,
 - generación de respuesta,
 - voz,
 - grafos semánticos.
7. No debes saltarte pasos.
8. Cuando terminemos cada fase, debes ayudarme a validar si el resultado quedó correcto.
9. Si el componente puede diseñarse de varias maneras, explícame cuál sería la opción más sencilla para un laboratorio y cuál sería una opción más robusta para producción.
10. Tu respuesta debe ayudarme a aprender, no solo a copiar código.

Comencemos con el paso actual del laboratorio:
 [PEGAR AQUÍ EL PASO EN EL QUE ESTÁS TRABAJANDO]

DESARROLLO DEL LABORATORIO:

Qué vamos a construir:

Capa 1. Comprensión documental

Leer PDF, Word, web, texto y eventualmente BD.

Capa 2. Conversación inteligente

Responder por chat con memoria conversacional y citas al documento.

Capa 3. Interacción por voz

Hablarle al sistema y recibir respuesta hablada.

Capa 4. Estructuración del conocimiento

Extraer entidades, conceptos, relaciones y construir **grafos semánticos**.

3. Arquitectura recomendada

3.1. Frontend

Para empezar rápido:

- **Streamlit** si quieres prototipo rápido
- **Gradio** si quieres demo fácil
- **FastAPI + React** si quieres arquitectura más profesional

Mi recomendación para una primera versión funcional:

- **FastAPI** como backend
 - **Streamlit** como interfaz inicial
 - luego migras a React si el proyecto crece
-

3.2. Backend Python

El backend debería separarse en módulos:

- ingestion/
- parsing/
- chunking/
- embeddings/
- vector_store/
- graph_store/
- retrieval/
- llm/
- chat/
- voice/
- api/

Esto evita que termine siendo un notebook gigante difícil de mantener.

3.3. Ingesta documental

Debes soportar al menos:

- PDF
- DOCX
- TXT
- HTML/Web
- Markdown
- CSV
- eventualmente base de datos

Librerías útiles:

- pymupdf para PDF
- python-docx para Word
- BeautifulSoup4 para web
- pandas para CSV/Excel
- unstructured si quieres parsing más robusto
- trafilatura si quieres extraer texto limpio desde web

Para PDF, te recomiendo primero:

- PyMuPDF (fitz) para extraer texto rápido
- OCR solo cuando el PDF sea imagen

3.4. Chunking

No hagas chunking solo por longitud fija.
Hazlo por estrategia.

Te recomiendo 3 niveles:

a) Chunking simple

Por ejemplo:

- 500 a 900 caracteres, o
- 150 a 300 palabras,
- con overlap del 10% al 20%

b) Chunking semántico

Separar por:

- títulos
- subtítulos
- párrafos
- tablas
- secciones

c) Chunking jerárquico

Guardar:

- documento
- sección
- subsección
- chunk

Esto sirve mucho para citar mejor y reconstruir contexto.

Mi recomendación inicial:

- chunk por secciones si el documento lo permite
- tamaño entre **300 y 800 tokens**
- overlap de **50 a 100 tokens**

3.5. Embeddings

Cada chunk se convierte en vector.

Opciones:

- sentence-transformers
- OpenAI embeddings
- Azure OpenAI embeddings
- modelos BGE
- intfloat/multilingual-e5
- BAAI/bge-m3

Como trabajas mucho en español, yo evaluaría:

- intfloat/multilingual-e5-base
- BAAI/bge-m3

Si quieres local:

- sentence-transformers
- transformers

Si quieres servicio administrado:

- OpenAI o Azure OpenAI

3.6. Base vectorial

Opciones reales:

- **FAISS**: excelente para local y prototipo
- **Chroma**: sencilla, buena para pruebas
- **Qdrant**: muy buena para producción
- **Weaviate**: potente
- **Pinecone**: SaaS cómodo

Mi recomendación:

- prototipo local: **FAISS o Chroma**
- crecimiento serio: **Qdrant**

Qdrant es muy buena opción para combinar:

- vectores
- filtros por metadatos
- escalabilidad razonable

3.7. Grafo semántico

Aquí está el salto de valor.

El grafo no reemplaza el vector store.
Lo complementa.

Qué guardas en el grafo

Nodos como:

- persona
- organización
- concepto
- norma
- proyecto
- sistema
- indicador
- tecnología
- documento
- sección

Relaciones como:

- “depende_de”
- “regula”
- “menciona”
- “implementa”
- “se_relaciona_con”
- “usa”
- “pertenece_a”
- “afecta_a”

Tecnologías

- **Neo4j** es la más práctica para empezar
- también puede ser networkx para prototipos pequeños
- si quieres solo visualización local: pyvis

Mi recomendación:

- para PoC: networkx + pyvis
- para versión seria: **Neo4j**

3.8. Retrieval híbrido

Aquí está la diferencia entre un sistema normal y uno bueno.

No recuperes solo por similitud vectorial.

Combina:

- búsqueda vectorial
- búsqueda léxica/keyword (BM25)
- recuperación por grafo
- re-ranking

Esto mejora mucho la precisión.

Estrategia

1. El usuario pregunta
2. Se genera embedding de la pregunta
3. Se buscan chunks en vector DB
4. Se buscan coincidencias léxicas
5. Se identifican entidades clave
6. Se consulta el grafo semántico
7. Se combinan resultados
8. Se re-ranquean
9. Se pasan al LLM

3.9. LLM

El LLM recibe:

- pregunta del usuario
- historial conversacional resumido
- chunks recuperados
- relaciones relevantes del grafo
- instrucciones de respuesta

Opciones:

- OpenAI
- Azure OpenAI
- modelos open source con Ollama o vLLM

Para prototipo rápido:

- OpenAI / Azure OpenAI

Para control local:

- Ollama con modelos compatibles

3.10. Voz

La capa de voz tiene dos partes:

Speech to Text

Convertir voz a texto:

- Whisper local
- OpenAI Whisper API
- Azure Speech to Text

Text to Speech

Convertir respuesta a audio:

- ElevenLabs
- Azure Speech
- gTTS para algo simple
- Coqui TTS si quieres local

Mi recomendación:

- **Whisper** para STT
- **Azure Speech** o **ElevenLabs** para TTS

4. Stack recomendado en Python

Te propongo este stack base:

Núcleo documental

- pymupdf
- python-docx
- beautifulsoup4
- pandas

NLP / embeddings

- sentence-transformers
- transformers
- spacy

Vector DB

- faiss-cpu o qdrant-client

Grafo

- networkx
- pyvis
- luego neo4j

Orquestación

- langchain o llama-index

API

- fastapi
- uvicorn

Interfaz

- streamlit

Voz

- openai-whisper o faster-whisper
- sounddevice / pyaudio
- edge-tts, azure-cognitiveservices-speech o ElevenLabs

5. Arquitectura lógica del sistema

Te la dejo en forma de módulos:

Módulo 1. Ingesta

Carga documentos y extrae texto.

Módulo 2. Preprocesamiento

Limpia texto, separa secciones, detecta idioma, metadatos.

Módulo 3. Indexación vectorial

Genera embeddings y almacena chunks.

Módulo 4. Extracción semántica

Extrae entidades y relaciones.

Módulo 5. Grafo

Construye nodos y aristas.

Módulo 6. Retrieval

Hace búsqueda híbrida.

Módulo 7. Generación

Envía contexto al LLM y produce respuesta.

Módulo 8. Conversación

Guarda historial y memoria.

Módulo 9. Voz

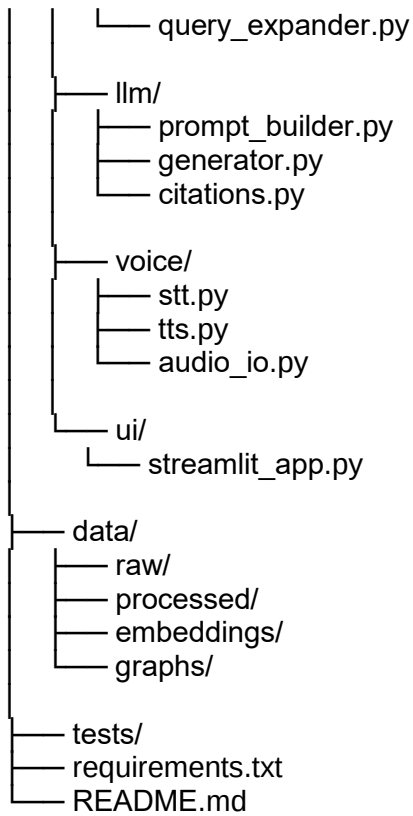
Procesa entrada de voz y salida hablada.

Módulo 10. Visualización

Muestra fuentes, trazabilidad y grafo.

6. Diseño de carpetas sugerido

```
rag_system/  
├── app/  
│   ├── main.py  
│   └── api/  
│       ├── routes_chat.py  
│       ├── routes_docs.py  
│       ├── routes_voice.py  
│       └── routes_graph.py  
├── core/  
│   ├── config.py  
│   ├── logger.py  
│   └── utils.py  
├── ingestion/  
│   ├── pdf_loader.py  
│   ├── docx_loader.py  
│   ├── web_loader.py  
│   └── db_loader.py  
├── processing/  
│   ├── cleaner.py  
│   ├── chunker.py  
│   ├── metadata.py  
│   └── entity_extractor.py  
├── embeddings/  
│   └── embedder.py  
├── vectorstore/  
│   ├── faiss_store.py  
│   └── qdrant_store.py  
├── graph/  
│   ├── graph_builder.py  
│   ├── neo4j_store.py  
│   └── visualizer.py  
└── retrieval/  
    ├── hybrid_retriever.py  
    └── reranker.py
```



7. Flujo técnico completo

Flujo de indexación

1. Subes documento
2. Extraes texto
3. Limpias contenido
4. Lo fragmentas
5. Generas embeddings
6. Guardas chunks + metadatos en vector DB
7. Extraes entidades y relaciones
8. Construyes/actualizas grafo

Flujo de consulta

1. Usuario escribe o habla
2. Si habla, STT convierte audio a texto
3. Se detectan entidades de la pregunta
4. Se hace retrieval híbrido
5. Se arma el prompt con evidencia
6. LLM responde
7. Se muestran fuentes
8. Opcional: TTS lee la respuesta

8. Qué metadatos debes guardar por chunk

Esto es clave.

Cada chunk debería tener:

- doc_id
- doc_name
- source_type
- page
- section
- chunk_id
- chunk_text
- tokens
- language
- created_at
- entities
- keywords

Esto luego te permite:

- citar bien
- filtrar por documento
- reconstruir respuestas
- enlazar con el grafo

9. Cómo construir los grafos semánticos

Hay dos enfoques.

Enfoque 1. NLP clásico

Con spaCy o modelos NER:

- extraes entidades
- detectas coocurrencias
- construyes relaciones simples

Ventaja:

- rápido
- controlado

Desventaja:

- menos profundo semánticamente

Enfoque 2. Extracción asistida por LLM

Le pides al LLM algo como:

“Extrae entidades y relaciones en formato JSON”

Ejemplo de salida:

```
{
  "entities": [
    {"name": "Power BI", "type": "Technology"},
    {"name": "Oracle", "type": "Database"},
    {"name": "Azure", "type": "Cloud"}
  ],
  "relations": [
    {"source": "Azure", "target": "Oracle", "relation": "connects_to"},
    {"source": "Power BI", "target": "Azure", "relation": "integrates_with"}
  ]
}
```

Ventaja:

- mayor riqueza semántica

Desventaja:

- costo
- posible ruido si no controlas bien

Mi recomendación:

- mezcla ambos
- NLP clásico para base estructural
- LLM para enriquecer relaciones complejas

10. Respuesta con trazabilidad

Tu sistema no debe responder “como si supiera”.
Debe responder con evidencia.

Ejemplo ideal:

- respuesta
- documentos usados
- páginas/secciones
- chunks relevantes
- entidades relacionadas
- confianza aproximada

Eso aumenta muchísimo la calidad.

11. Librerías concretas que sí usaría

Para una primera versión seria:

```
pip install fastapi uvicorn streamlit pymupdf python-docx beautifulsoup4 pandas  
pip install sentence-transformers faiss-cpu rank-bm25 spacy networkx pyvis  
pip install faster-whisper openai
```

Si usas Neo4j:

```
pip install neo4j
```

Si usas Qdrant:

```
pip install qdrant-client
```

12. Ruta de implementación por fases

Fase 1. MVP documental

Objetivo: conversar por chat con PDFs y DOCX

Incluye:

- carga de documentos
- extracción de texto
- chunking
- embeddings
- FAISS
- chat básico con citas

Fase 2. Retrieval mejorado

Objetivo: mejorar precisión

Incluye:

- BM25
- retrieval híbrido
- filtros por metadatos
- re-ranking

Fase 3. Voz

Objetivo: hablar con los documentos

Incluye:

- STT con Whisper

- TTS
- botón de micrófono
- respuesta hablada

Fase 4. Grafo semántico

Objetivo: estructurar conocimiento

Incluye:

- NER
- extracción de relaciones
- networkx/Neo4j
- visualización del grafo

Fase 5. Producción

Objetivo: sistema robusto

Incluye:

- FastAPI
- autenticación
- colas
- persistencia
- monitoreo
- despliegue cloud

13. Recomendación técnica honesta

Para no complicarte desde el inicio, yo haría esto:

MVP recomendado

- Python
- FastAPI
- Streamlit
- PyMuPDF
- sentence-transformers
- FAISS
- OpenAI o Azure OpenAI
- faster-whisper
- networkx + pyvis

Con eso puedes tener:

- chat documental
- voz
- visualización de grafo

- arquitectura entendible y extensible

Luego evolucionas a:

- Qdrant
- Neo4j
- reranker
- multiusuario
- base de datos relacional para sesiones

14. Riesgos que debes evitar

Error 1

Hacer chunking ciego por tamaño fijo.

Error 2

No guardar metadatos.

Error 3

Usar solo vector search.

Error 4

No citar las fuentes.

Error 5

Construir un grafo “bonito” pero no útil.

Error 6

Meter todo en un notebook.

Error 7

No separar indexación y consulta.

15. Qué te recomiendo como siguiente paso

Yo te sugiero construirlo así:

Paso 1. MVP de chat documental

Paso 2. añadir voz

Paso 3. añadir extracción de entidades y grafo

Paso 4. pasar a retrieval híbrido

Paso 5. endurecer arquitectura para producción

16. Prompt técnico de diseño del proyecto

Te dejo un prompt base muy bueno para seguir desarrollándolo:

Actúa como arquitecto senior de soluciones RAG en Python.

Quiero diseñar una aplicación para conversar con documentos mediante chat y voz, incorporando además grafos semánticos derivados del contenido documental.

La solución debe incluir:

1. Carga de documentos PDF, DOCX, TXT y páginas web.
2. Extracción y limpieza de texto.
3. Fragmentación inteligente (chunking) con metadatos.
4. Generación de embeddings.
5. Almacenamiento en base vectorial.
6. Recuperación híbrida (vectorial + léxica + semántica).
7. Uso de un LLM para responder con base en evidencia documental.
8. Interfaz de chat.
9. Entrada y salida por voz.
10. Construcción y visualización de grafos semánticos de entidades y relaciones.

Diseña:

- arquitectura lógica
- arquitectura física
- estructura de carpetas
- stack tecnológico recomendado
- flujo de indexación
- flujo de consulta
- diseño de metadatos
- estrategia de chunking
- estrategia de retrieval
- estrategia para extracción de entidades y relaciones
- ejemplo de endpoints FastAPI
- ejemplo de clases Python
- recomendaciones para pasar de MVP a producción

Usa un enfoque modular, escalable y claro.

Numeral	Fase / paso del laboratorio	Propósito del paso	Prompt de apoyo actualizado
1	Comprender la arquitectura general del sistema	Reconocer la lógica de un sistema RAG y entender la función de cada componente antes de programar.	Actúa como tutor experto en arquitectura RAG para fines educativos. Explicame de forma técnica, clara y pedagógica la arquitectura general de un sistema básico para conversar con documentos. Quiero comprender el papel de cada componente: documento, extracción de texto, chunking, embeddings, base vectorial, retrieval, LLM, chat, voz y grafo semántico. Indícame cómo se relacionan entre sí, qué dependencias existen y qué errores conceptuales debo evitar. Enfócate en una versión comprensible para un laboratorio académico, no en una arquitectura empresarial completa.
2	Preparar el entorno de trabajo	Instalar, organizar y validar las librerías necesarias para desarrollar el MVP del laboratorio.	Actúa como tutor experto en Python para IA aplicada. Ayúdame a preparar el entorno de trabajo para construir un laboratorio RAG paso a paso. Necesito una lista mínima de librerías para un MVP académico, la función de cada una, un bloque de instalación ordenado y una rutina breve para verificar que todas quedaron correctamente instaladas. Diferencia cuáles son obligatorias para el laboratorio y cuáles podrían dejarse para una fase posterior.
3	Cargar y extraer texto desde documentos	Obtener texto desde archivos PDF o DOCX como insumo inicial del sistema.	Actúa como tutor experto en ingesta documental con Python. Ayúdame a construir el paso de carga y extracción de texto desde un documento PDF o DOCX en una versión sencilla para laboratorio. Explicame primero el propósito del paso, luego dame código comentado y finalmente indícame cómo validar si la

Numeral	Fase / paso del laboratorio	Propósito del paso	Prompt de apoyo actualizado
			extracción fue correcta. Señálame errores comunes, como PDFs escaneados, saltos de línea anómalos o pérdida de caracteres, y cómo detectarlos.
4	Limpiar y fragmentar el documento	Transformar el texto extraído en fragmentos útiles para su vectorización y posterior recuperación.	Actúa como tutor experto en procesamiento de texto para RAG. Ayúdame a limpiar el texto extraído y a realizar chunking en Python en una forma sencilla pero correcta para un laboratorio. Quiero entender qué es fragmentar, por qué el tamaño del fragmento afecta la recuperación y cómo implementar un chunking claro y funcional. Explícame el criterio usado, dame código comentado y sugiéndome cómo validar si los chunks quedaron bien contruidos.
5	Generar embeddings	Convertir cada fragmento textual en una representación numérica que capture su significado semántico.	Actúa como tutor experto en embeddings para sistemas RAG. Explícame qué son los embeddings, para qué sirven en este laboratorio y cómo generarlos en Python a partir de una lista de fragmentos. Indícame qué modelo conviene usar en una práctica académica sencilla, qué entrada recibe el proceso, qué salida genera y cómo verificar que los vectores producidos sean correctos y consistentes. Prioriza una opción fácil de implementar en laboratorio.
6	Almacenar embeddings en una base vectorial	Crear un índice vectorial básico que permita recuperar fragmentos similares a una consulta.	Actúa como tutor experto en bases vectoriales para laboratorio. Ayúdame a almacenar los embeddings de mis fragmentos en una base vectorial sencilla, como FAISS o Chroma, priorizando la opción más fácil para una práctica guiada. Explícame qué hace una base vectorial, cómo se construye

Numeral	Fase / paso del laboratorio	Propósito del paso	Prompt de apoyo actualizado
			el índice, cómo se asocia cada vector a su fragmento y metadatos, y cómo validar que el almacenamiento fue exitoso. Quiero código comentado y una explicación clara de las decisiones tomadas.
7	Recuperar fragmentos relevantes a partir de una pregunta	Consultar la base vectorial para encontrar los fragmentos más cercanos semánticamente a la pregunta del usuario.	Actúa como tutor experto en retrieval para RAG. Ayúdame a implementar el paso de recuperación semántica en una versión mínima funcional para laboratorio. Quiero tomar una pregunta del usuario, transformarla en embedding y buscar los fragmentos más cercanos dentro de mi base vectorial. Explicame el flujo completo, dame código comentado y enséñame cómo interpretar los resultados obtenidos. Indícame además cómo decidir cuántos fragmentos recuperar en una práctica sencilla.
8	Generar una respuesta fundamentada en el contexto recuperado	Construir un prompt robusto para que el LLM responda usando solo la evidencia recuperada del documento.	Actúa como tutor experto en diseño de prompts para sistemas RAG. Ayúdame a construir el prompt final que se enviará al modelo de lenguaje para una práctica académica. Quiero que el modelo responda usando únicamente el contexto recuperado del documento, evitando inventar información. Explicame la estructura recomendada del prompt, dame una plantilla comentada y sugiérme mecanismos sencillos para reducir alucinaciones y mejorar la trazabilidad de la respuesta.
9	Incorporar interacción por voz	Permitir que el usuario pregunte hablando y, si es	Actúa como tutor experto en integración de voz para aplicaciones de IA. Ayúdame a

Numeral	Fase / paso del laboratorio	Propósito del paso	Prompt de apoyo actualizado
		posible, que el sistema responda también con voz en una versión simple.	extender mi laboratorio RAG con una capa básica de voz. Quiero comprender el flujo de Speech-to-Text y Text-to-Speech dentro del sistema, pero en una versión sencilla para laboratorio. Dime qué herramientas usar, cómo conectarlas con el flujo RAG y qué limitaciones técnicas debo tener en cuenta antes de implementar.
10	Explorar grafos semánticos a partir del contenido	Representar entidades y relaciones del documento en una estructura de conocimiento complementaria al vector store.	Actúa como tutor experto en grafos semánticos aplicados a documentos. Ayúdame a construir una aproximación inicial para identificar entidades, detectar relaciones y representar esa información en un grafo usando Python. Quiero una estrategia sencilla para laboratorio, una explicación del valor del grafo dentro de una arquitectura documental y una forma básica de visualizarlo. Señálame también las diferencias entre un grafo semántico y una base vectorial.
11	Documentar, evaluar y reflexionar sobre el sistema construido	Analizar críticamente el prototipo desarrollado, justificar decisiones técnicas y reconocer mejoras futuras.	Actúa como tutor experto en evaluación técnica de sistemas RAG. Ayúdame a redactar una reflexión final sobre el sistema construido en este laboratorio. Quiero identificar qué funcionó bien, qué limitaciones encontré, qué errores surgieron, qué decisiones técnicas tomé y cómo podría evolucionar este prototipo hacia una versión más robusta. Organiza la respuesta en lenguaje académico y orientado a documentación técnica de laboratorio.